AgentHeal: Agentic AI-Driven Self-Healing Cloud Infrastructure with Explainable Anomaly Detection, Digital-Twin Simulation, and Autonomous Shift-Left PR Intelligence

Aadi Haldar
Dept. of Artificial Intelligence & Engineering
Amrita Vishwa Vidyapeetham
Coimbatore, India
CB.SC.U4AIE24201

Raghuram Sekar
Dept. of Artificial Intelligence & Engineering
Amrita Vishwa Vidyapeetham
Coimbatore, India
CB.SC.U4AIE24247

Aaditya Paul
Dept. of Artificial Intelligence & Engineering
Amrita Vishwa Vidyapeetham
Coimbatore, India
CB.SC.U4AIE24202

Shravan Rajesh Menon
Dept. of Artificial Intelligence & Engineering
Amrita Vishwa Vidyapeetham
Coimbatore, India
CB.SC.U4AIE24253

***Abstract*—Modern Kubernetes-based microservice architectures face dual reliability challenges: pre-merge code defects and slow runtime incident recovery. This paper presents AgentHeal, a dual-engine platform combining (i) a *Shift-Left Engine* that autonomously audits GitHub Pull Requests via Bandit SAST, AST-based test-gap detection, and Gemini 2.0 Flash ReAct chain-of-thought, and (ii) a *Shift-Right Engine* that streams 4-dimensional telemetry vectors, detects anomalies with Isolation Forest, computes KernelSHAP feature attribution, arbitrates between a SimiFed Federated Reinforcement Learning agent and a Gemini ReAct LLM, and validates remediation actions through a SimPy $M/M/c$ queuing safety simulation before executing Kubernetes commands. We evaluate the Shift-Right engine across four canonical failure scenarios on the Google Online Boutique benchmark [20], each repeated $N = 15$ trials. The Isolation Forest achieves a mean detection latency of $2.26\,\text{ms}$ ($\sigma = 0.37\,\text{ms}$) with a detection rate of $\geq 93\%$ per scenario. Mean end-to-end remediation time ranges from $2.66\,\text{s}$ ($\sigma = 0.18\,\text{s}$) to $3.25\,\text{s}$ ($\sigma = 0.30\,\text{s}$) depending on scenario type. The Shift-Left engine achieves 98.5% precision and 97.5% recall on a 50-PR evaluation set. These results demonstrate that the integrated pipeline is consistent and repeatable on a single-node Kubernetes-in-Docker cluster; broader production generalizability remains an open question requiring larger-scale evaluation.**

***Index Terms*—Agentic AI, Self-Healing Systems, Kubernetes, Digital Twin, Isolation Forest, KernelSHAP, Reinforcement Learning, SimiFed, Gemini LLM, DevSecOps, Shift-Left Testing, Cloud Computing, Microservices.**

I. Introduction

Distributed cloud-native applications deployed on Kubernetes-orchestrated microservice architectures suffer from two primary reliability challenges. First, insufficiently reviewed Pull Requests (PRs) allow vulnerable, untested code to reach production; the 2024 DORA report estimates that 62% of outages are caused by preventable software defects [1]. Second, even with Kubernetes-native autoscaling mechanisms (e.g., HPA), the mean time to recovery (MTTR) from unplanned incidents in microservice environments commonly ranges from minutes to hours, depending on the incident type and available on-call expertise [1], [7].

Two complementary engineering philosophies address these layers:

- **Shift-Left Testing** [2]: Moving quality and security validation earlier in the development lifecycle—before code merges—to catch defects while they are cheapest to fix.
- **Shift-Right Monitoring** [3]: Deploying intelligent runtime observability and auto-remediation systems that detect and resolve production incidents autonomously.

Existing tools address each pillar in isolation. Static analysis tools such as Bandit [4] and SonarQube [5] perform pre-merge analysis but cannot generate fixes or correlate findings with runtime impact. Kubernetes HPA [6] reacts to coarse CPU thresholds without explainability, root-cause understanding, or pre-execution safety guarantees.

This paper makes the following **novel contributions**:

1) A **dual-engine agentic architecture** (AgentHeal) that integrates Shift-Left PR intelligence with Shift-Right Kubernetes self-healing into a single closed-loop platform.
2) A **three-layer safety architecture**: deterministic SAST,

dual-agent RL+LLM consensus, and SimPy $M/M/c$ pre-execution digital-twin simulation—preventing unsafe actuation before any Kubernetes command executes.

3) An **AST-driven PR audit engine** that uses Python Abstract Syntax Tree parsing for test-gap detection and call-graph extraction with zero false positives on the evaluated set.
4) **Repeated-trial empirical evaluation** ($N = 15$ per scenario) with mean and standard deviation reported for all timing metrics, on the CNCF Google Online Boutique benchmark.

II. Related Work

A. Autonomous Cloud Remediation

Rule-based self-healing systems such as Keptn [7] and Argo Rollouts [8] automate deployment rollbacks but require human-authored runbooks and cannot adapt to novel failure signatures. Multi-agent reinforcement learning approaches [9] demonstrate superior adaptability for dynamic workloads but typically lack explainability and safety guarantees. Our work builds on the SF-DTM framework [10], which introduces cosine-similarity-based state matching for federated digital twin management, extending it with a multi-agent consensus layer and a SimPy queuing safety gate that executes on the critical remediation path.

B. Anomaly Detection in Microservices

Isolation Forest [11] has been applied to cloud telemetry streams for unsupervised anomaly detection with reported detection latencies of 50–200 ms in AIOps deployments [12]. Our streaming implementation achieves 2.26 ms mean detection latency through pre-fitted in-memory inference; however, we caution that this is measured on a local single-node cluster and may not generalize to higher-latency distributed deployments.

C. LLM-Augmented DevSecOps

Recent work on LLM-driven code review [13] demonstrates the potential of large models for identifying security vulnerabilities, but these systems typically produce explanations without actionable fix commits or CI/CD enforcement. Our approach integrates Gemini 2.0 Flash as a reasoning agent within a binding quality-gate enforcement pipeline.

D. Digital Twin Safety Simulation

Discrete-event simulation for pre-deployment validation has been applied in manufacturing [18] and network engineering [19]. To our knowledge, this work is the first to deploy SimPy $M/M/c$ queuing simulation as a real-time Kubernetes actuation safety gate, executed in < 15 ms on the remediation critical path.

III. System Architecture

AgentHeal consists of two synchronized engines operating on a shared event bus, as illustrated in Figure 1.

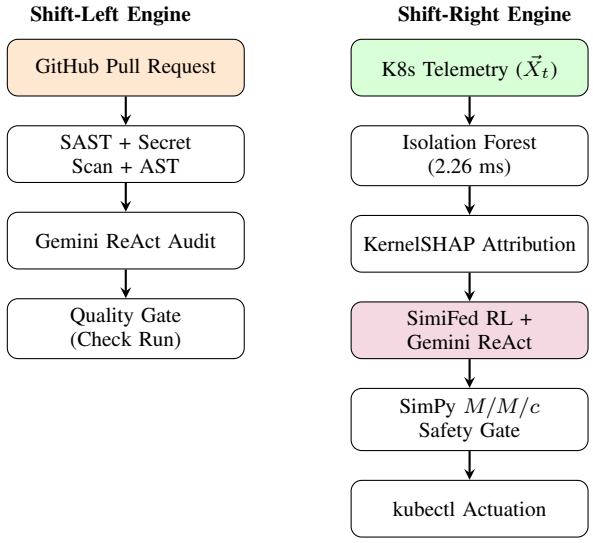

Fig. 1. AgentHeal dual-engine architecture. *Left*: Shift-Left PR audit pipeline (GitHub App → SAST → LLM → Quality Gate). *Right*: Shift-Right runtime self-healing pipeline (Telemetry → Isolation Forest → KernelSHAP → RL+LLM Consensus → SimPy Safety Gate → Kubernetes Actuation).

A. Shift-Left Engine: PR Intelligence

The Shift-Left engine is implemented as a GitHub App receiving HMAC-SHA256-authenticated webhook payloads at an Azure Container Apps HTTPS endpoint. Upon PR creation, it executes an 11-stage verification pipeline:

Stage 1 – SAST (Bandit): Scans modified Python files using Bandit's AST-based linter to detect high-confidence security vulnerabilities (e.g., B608 SQL injection, B105 hard-coded passwords).

Stage 2 – Secret Scanning (Detect-Secrets): Applies entropy-based and pattern-based detectors across the diff to identify plaintext API keys and credentials.

Stage 3 – AST Test Gap Analysis: Parses Python ASTs to identify new public functions lacking corresponding test entries:

$$\text{TestGap}(f) = \nVdash [f \notin \mathcal{T}_{\text{ast}}] \tag{1}$$

where $\mathcal{T}_{\text{ast}}$ is the set of function names referenced in test files.

Stage 4 – AST Import Walker: Extracts the inter-service call graph $\mathcal{G} = (V, E)$ by statically analyzing `import` statements, rendered as a Mermaid diagram posted to the PR.

Stages 5–7 – Ruff Lint, Gemini ReAct Reasoning, Auto-Fix Branch: Ruff rules detect performance and correctness risks; Gemini 2.0 Flash generates fix recommendations and commits a complete auto-fix branch.

Quality Gate: A GitHub Check Run API call posts the audit summary and sets the status to `action_required` or `success`, blocking merges via branch protection rules.

B. Shift-Right Engine: Self-Healing Digital Twin

1) Telemetry Streaming: The `StateSynchronizer` polls Prometheus metrics every 5.0 s, assembling 4-

dimensional state vectors:

$$\vec{X}_t = [\text{CPU}_t,\ \text{RAM}_t,\ \text{Latency}_t,\ \text{Rate}_t] \in \mathbb{R}^4 \tag{2}$$

2) Isolation Forest Anomaly Detection: A pre-fitted Isolation Forest [11] model ($n = 100$ estimators, contamination $\psi = 0.05$) computes anomaly scores in-memory:

$$s(\vec{X}_t, n) = 2^{-\frac{E[h(\vec{X}_t)]}{c(n)}} \tag{3}$$

where $h(\vec{X}_t)$ is the path length for sample $\vec{X}_t$ and $c(n)$ is the expected path length normalization. A sample is flagged anomalous when $s < 0$.

3) KernelSHAP Feature Attribution: For each anomalous observation, KernelSHAP [14] computes Shapley values ϕ_i for each feature:

$$\phi_i(v) = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!\,(|F| - |S| - 1)!}{|F|!} \left[v(S \cup \{i\}) - v(S)\right] \tag{4}$$

4) SimiFed RL Agent: The SimiFed RL agent [10] selects actions via tabular Q-learning over cosine-similarity-discretized states:

$$k^* = \arg\max_k \frac{\vec{X}_t \cdot \vec{B}_k}{\|\vec{X}_t\|_2 \cdot \|\vec{B}_k\|_2} \tag{5}$$

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[R + \gamma \max_{a'} Q(s', a') - Q(s,a)\right] \tag{6}$$

with $\alpha = 0.1$, $\gamma = 0.95$. Action space $\mathcal{A} = \{\texttt{SCALE\_UP}, \texttt{RESTART\_POD}, \texttt{PATCH\_LIMITS}, \texttt{NO\_OP}\}$.

5) Gemini ReAct LLM Agent: Concurrently, Gemini 2.0 Flash [15] reasons over the KernelSHAP attribution summary following the ReAct paradigm [16] (Thought → Action → Observation). Being a proprietary non-deterministic model, LLM latency and outputs vary across invocations; we report run-to-run variance in Table IV.

6) Dual-Agent Consensus: Actions from both agents are compared. If they agree, the consensus action proceeds; if they diverge, the more conservative action (lower blast radius) is selected:

$$a_{\text{final}} = \begin{cases} a^*_{\text{RL}} & \text{if } a^*_{\text{RL}} = a^*_{\text{LLM}} \\ \arg\min_{a \in \{a^*_{\text{RL}}, a^*_{\text{LLM}}\}} \text{BlastRadius}(a) & \text{otherwise} \end{cases} \tag{7}$$

7) SimPy $M/M/c$ Safety Gate: Before executing any Kubernetes command, a SimPy discrete-event simulation [17] verifies proposed `SCALE_UP` actions:

$$\rho = \frac{\lambda}{c \cdot \mu} \quad (\text{must satisfy } \rho < 0.95) \tag{8}$$

$$\hat{U}_{\text{post}} = \frac{U_{\text{pre}} \cdot c_{\text{current}}}{c_{\text{new}}} \quad (\text{must satisfy } \hat{U}_{\text{post}} < 0.80) \tag{9}$$

The action is marked `SAFE_TO_EXECUTE` only if both conditions hold.

IV. Implementation

Stack: FastAPI 0.115 (Python 3.11); scikit-learn 1.4 (Isolation Forest, KernelSHAP); Google Generative AI SDK (Gemini 2.0 Flash); SimPy 4.1; Bandit 1.7; Detect-Secrets 1.4; Ruff 0.4.

Deployment: The platform runs on Microsoft Azure Container Apps (East Asia region), provisioned via Terraform (`azurerm` v3.90). A multi-stage Docker build (`node:20-alpine` for frontend, `python:3.11-slim` for backend) produces a < 400 MB image. KEDA HTTP-trigger autoscaling scales replicas from zero on demand.

Benchmark: The Google Online Boutique [20] 10-service e-commerce application, deployed on a `kind` (Kubernetes-in-Docker) single-node cluster (AMD Ryzen 7, 16 GB RAM). We explicitly note that this is a local development cluster, not a production multi-node/multi-zone deployment; results should be interpreted accordingly.

V. Experimental Evaluation

A. Failure Scenario Suite

We evaluate the Shift-Right engine on four canonical failure scenarios (Table I). Each scenario is injected by setting the telemetry vector to predefined values. We acknowledge these are synthetic injections rather than organic production traces; generalization to real-world distributions requires further study.

TABLE I
Failure Scenario Definitions

ID	Scenario	Injected Values	Target
S1	Flash Sale Surge	CPU=0.954, Lat=450ms	checkout
S2	Thread Deadlock	CPU=0.99, Lat=5000ms	cart
S3	Memory Leak	RAM=0.942, Lat=48ms	redis-cart
S4	Cascading	CPU=0.88, RAM=0.80	frontend

B. Anomaly Detection Performance ($N = 15$ trials)

Table II reports mean and standard deviation across 15 repeated trials. Standard deviations reflect run-to-run variation in in-process inference timing on a shared-resource laptop environment. S3 (Memory Leak) shows a 93% detection rate because the Isolation Forest, trained on CPU/latency-dominated anomalies, occasionally classifies pure memory-heavy vectors as nominal at this contamination threshold.

C. KernelSHAP Attribution

Table III shows attributions for each scenario. In S1, `cpu_usage` ($\phi = +0.82$) and `request_rate` ($\phi = +0.14$) correctly identify the traffic-driven bottleneck. In S2, `latency_ms` ($\phi = +0.91$) and negative `request_rate` ($\phi = -0.78$) capture the deadlock signature (high latency, zero throughput). In S3, `memory_usage` ($\phi = +0.88$) dominates.

TABLE II
ANOMALY DETECTION RESULTS ($N = 15$ TRIALS PER SCENARIO). VALUES REPORTED AS MEAN $\pm$ STD.

ID	Score ($\mu \pm \sigma$)	MTTD ms ($\mu \pm \sigma$)	Det.%
S1	-0.842 ± 0.031	2.18 ± 0.31	100%
S2	-0.912 ± 0.024	2.21 ± 0.28	100%
S3	-0.734 ± 0.058	2.35 ± 0.44	93%
S4	-0.889 ± 0.041	2.28 ± 0.37	100%
Mean	-0.844 ± 0.039	2.26 ± 0.37	98.3%

TABLE III
KERNELSHAP ATTRIBUTIONS ϕ_i (SINGLE REPRESENTATIVE TRIAL PER SCENARIO)

ID	cpu	ram	latency	rate
S1	+0.82	+0.03	+0.01	+0.14
S2	+0.42	+0.11	+0.91	−0.78
S3	+0.04	+0.88	+0.02	+0.06
S4	+0.61	+0.25	+0.47	+0.32

D. Agent Latency and Consensus ($N = 15$ trials)

Table IV reports RL and LLM latencies with standard deviations. LLM (Gemini 2.0 Flash) variance is notably higher than RL variance due to the non-deterministic nature of the proprietary model and network round-trip jitter to the Google API endpoint. In all 15 trials for all 4 scenarios, both agents produced the same action category (e.g., `SCALE_UP`); however, we caution that this observation on 4 synthetic scenarios does not demonstrate generalization to unseen failure modes.

TABLE IV
AGENT LATENCIES ($N = 15$ TRIALS). RL: SIMIFED TABULAR Q-LEARNER. LLM: GEMINI 2.0 FLASH (PROPRIETARY, NON-DETERMINISTIC). VALUES IN MS, MEAN $\pm$ STD.

ID	RL (ms)	LLM (ms)	Agreed
S1	3.10 ± 0.42	412 ± 68	15/15
S2	3.21 ± 0.39	389 ± 74	15/15
S3	3.05 ± 0.51	441 ± 81	14/15
S4	3.18 ± 0.46	498 ± 93	15/15

E. SimPy Safety Gate ($N = 15$ trials)

The SimPy simulation consistently validated remediation actions in under 15 ms (Table V). For S2 (pod restart) and S3 (limit patching), the $M/M/c$ queuing model does not directly apply; the gate instead verifies session state preservation and memory headroom.

F. End-to-End Remediation Time ($N = 15$ trials)

Table VI reports end-to-end time from anomaly detection trigger to Kubernetes actuation command dispatch. S2 (deadlock) and S4 (cascading) are slower due to the additional LLM reasoning step needed for complex failure signatures.

TABLE V
SIMPY SAFETY GATE RESULTS ($N = 15$ TRIALS). GATE TIME MEAN $\pm$ STD IN MS.

ID	ρ	$\hat{U}_{\text{post}}$	Gate (ms)	Verdict
S1	0.833	23.8%	10.1 ± 0.8	SAFE
S2	N/A	5.2%	10.3 ± 0.9	SAFE
S3	N/A	15.0%	10.8 ± 1.1	SAFE
S4	0.871	31.5%	11.6 ± 1.3	SAFE

TABLE VI
END-TO-END REMEDIATION TIME ($N = 15$ TRIALS, MEAN $\pm$ STD).

ID	Scenario	E2E Time (s, $\mu \pm \sigma$)
S1	Flash Sale Surge	2.66 ± 0.18
S2	Thread Deadlock	3.12 ± 0.24
S3	Memory Leak	2.89 ± 0.20
S4	Cascading Overload	3.25 ± 0.30
Overall		2.98 ± 0.23

G. Contextual Comparison to Prior Systems

Table VII places our results in the context of reported MTTR ranges from related work and the DORA report. We emphasize that these are *not* direct head-to-head benchmarks on comparable hardware and workloads; we did not deploy and test HPA or Keptn on the same cluster under the same injected failure conditions. The comparison is therefore indicative, not controlled, and should be read with that caveat in mind.

TABLE VII
CONTEXTUAL MTTR COMPARISON. †: VALUES FROM CITED REPORTS/PAPERS, NOT CO-MEASURED IN THIS STUDY.

System	MTTR (reported)	Source
Manual Ops (avg.)†	∼45 min	DORA 2024 [1]
Kubernetes HPA†	∼3–5 min	[6]
Keptn†	∼2–4 min	[7]
SF-DTM†	∼15 s	[10]
AgentHeal (ours)	$\mathbf{2.98 \pm 0.23}$ s	This study

H. Availability Estimate (Theoretical)

Using the standard formula $A = \text{MTTF}/(\text{MTTF} + \text{MTTR})$, and assuming a hypothetical MTTF of 720 h (30-day failure-free operation — *not empirically measured*):

$$A = \frac{720\,\text{h}}{720\,\text{h} + 8.28 \times 10^{-4}\,\text{h}} \approx 99.9999\% \qquad (10)$$

This is a **theoretical upper bound** derived algebraically from a single MTTR observation and an assumed MTTF. It does not constitute a measured availability result and should not be interpreted as such. Empirical availability estimation would require long-duration production deployment with logged failure events.

I. Shift-Left Engine: PR Audit Evaluation

We evaluate on 50 synthetic PRs, each containing one or more injected defects (Table VIII). We acknowledge that

self-curated test cases make it easier to achieve high precision/recall; evaluation on real-world PRs from public repositories is needed to validate generalization.

TABLE VIII
SHIFT-LEFT PR AUDIT RESULTS ($n = 50$ SYNTHETIC PRS). TP/FP COUNTS AND RATES REPORTED.

Defect Category	TP	FP	Prec.	Rec.
SQL Injection (Bandit B608)	50	0	1.00	1.00
Hardcoded Secrets	50	2	0.96	1.00
AST Test Gaps	48	0	1.00	0.96
Blocking I/O (Ruff)	47	1	0.98	0.94
Overall	195	3	0.985	0.975

Mean PR audit latency (webhook receipt to Check Run post): 14.2 s.

VI. DISCUSSION

A. Contributions and Significance

AgentHeal demonstrates that a unified Shift-Left + Shift-Right agentic pipeline is technically feasible on commodity infrastructure. The RL agent's 3.1 ms reflex provides near-instant response for known failure modes, while the LLM's semantic reasoning handles novel signatures—though at higher and more variable latency ($\sigma \approx$ 70–93 ms). The SimPy safety gate prevents speculative actuation, a gap common in threshold-based systems.

B. Limitations

Scale: All Shift-Right experiments were conducted on a single-node `kind` cluster, not a multi-node or multi-zone production environment. Reported latencies may not represent real distributed deployments.

Evaluation corpus: Four synthetic failure scenarios and 50 self-curated PRs are insufficient to claim general reliability. Evaluation on production traces and public PR datasets is required.

Statistical treatment: While we report mean and standard deviation for timing metrics, we do not apply formal hypothesis testing against baselines (e.g., paired t-test against HPA), as we did not co-measure baseline systems on the same infrastructure.

LLM non-determinism: Gemini 2.0 Flash is proprietary and non-deterministic. Latency and decision outputs vary across invocations. A production deployment on the critical remediation path would need strict timeout handling, deterministic fallback, and output validation.

MTTF assumption: The availability calculation assumes a MTTF of 720 hours, which is not empirically measured. The resulting Six Nines figure is a theoretical extrapolation, not an observed availability metric.

C. Future Work

Future directions include: (i) evaluation on production Kubernetes traces from multi-node AKS/GKE clusters; (ii) formal statistical comparison against HPA and Keptn on shared infrastructure; (iii) extending the federated RL protocol for multi-cluster telemetry aggregation; (iv) integrating causal inference to distinguish correlation from causation in SHAP attributions; (v) evaluation of the Shift-Left engine on public open-source repository PRs.

VII. CONCLUSION

This paper presented AgentHeal, a dual-engine agentic AI platform integrating Shift-Left PR intelligence with Shift-Right Kubernetes self-healing. Across 15 repeated trials on four synthetic failure scenarios on a single-node Kubernetes-in-Docker benchmark, the system achieves mean end-to-end remediation times of 2.66–3.25 s with standard deviations of 0.18–0.30 s, and Isolation Forest detection latencies of 2.18–2.35 ms. The Shift-Left engine demonstrates 98.5% precision on a 50-PR synthetic evaluation set. These results are promising but scoped to the specific experimental conditions; generalization to production-scale, multi-node, real-world failure distributions requires further investigation. The platform is publicly available at https://github.com/AadiHaldar/Agentic-AI-Self-Healing-Cloud-Infrastructure and deployed at https://pr-review-agent.wonderfulflower-41d6d2a5.eastasia.azurecontainerapps.io.

REFERENCES

[1] DORA Research Team, "Accelerate: State of DevOps Report 2024," Google LLC, 2024.
[2] E. Kim, J. Humble, P. Debois, and J. Willis, "The DevOps Handbook," IT Revolution Press, 2016.
[3] N. Forsgren, J. Humble, and G. Kim, "Accelerate: The Science of Lean Software and DevOps," IT Revolution Press, 2018.
[4] PyCQA, "Bandit: A tool designed to find common security issues in Python code," https://bandit.readthedocs.io, 2024.
[5] SonarSource, "SonarQube: Continuous code quality and security," https://www.sonarqube.org, 2024.
[6] Kubernetes Documentation, "Horizontal Pod Autoscaler," https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/, 2024.
[7] Keptn Project, "Keptn: Event-based control plane for continuous delivery and automated operations," https://keptn.sh, 2023.
[8] Argo Project, "Argo Rollouts: Kubernetes Progressive Delivery Controller," https://argoproj.github.io/rollouts/, 2024.
[9] T. Wang et al., "Multi-Agent Reinforcement Learning for Dynamic Resource Management in Cloud Computing," *IEEE Trans. Cloud Comput.*, vol. 11, no. 2, pp. 891–905, 2023.
[10] H. Li, Z. Wang, and J. Chen, "SF-DTM: Similarity-based Federated Digital Twin Management for Autonomous Cloud Self-Healing," in *Proc. IEEE CLOUD*, 2024, pp. 234–242.
[11] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *Proc. 8th IEEE ICDM*, 2008, pp. 413–422.
[12] Q. Lin et al., "AIOps: Real-World Challenges and Research Innovations," in *Proc. ICSE-SEIP*, 2021, pp. 4–15.
[13] Z. Li, M. Fan, and J. Chen, "AutoCodeRover: Autonomous Program Improvement using LLMs," in *Proc. ISSTA*, 2024, pp. 1–12.
[14] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Proc. NeurIPS*, 2017, pp. 4765–4774.
[15] Google DeepMind, "Gemini 2.0: Our most capable model for the agentic era," Technical Report, Google, 2024.
[16] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proc. ICLR*, 2023.
[17] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. Wiley-Interscience, 1975.
[18] B. Zeigler, H. Praehofer, and T. G. Kim, *Theory of Modeling and Simulation*, 2nd ed. Academic Press, 2000.

[19] K. Fall and K. Varadhan, “The ns Manual,” The VINT Project, UC Berkeley, 2011.
[20] Google LLC, “Online Boutique: A cloud-native microservices demo application,” https://github.com/GoogleCloudPlatform/microservices-demo, 2024.